

Dokan

One Shelf for Every Kind of Thing

Object-Oriented Programming — Lab 13 — Generics

Marks: 10 **Language:** Java (JDK 17) **Build:** Gradle **Approach:** fill in a starter project, test-driven
5 classes, 2 interfaces. You get a running project with the work marked TODO, 27 worked test cases, and 41 unit tests.

What this lab teaches

Six ideas, and one small class for each. By the end you should be able to explain:

- a **type parameter** — `Box<T>` — and why it beats storing `Object`;
- **two** type parameters at once — `Pair<A,B>` — and a method that swaps them;
- a **bounded** type parameter — `Shelf<T extends Item>` — and what the bound buys you;
- an **unbounded** one — `Counter<T>` — and when you should *not* add a bound;
- **static generic methods** and **wildcards** — `? extends` and `? super`;
- **erasure**: what the compiler throws away, and the three things you therefore cannot write.

1. Generics in Five Minutes

Read this even if you think you know it. Everything in the lab is a consequence of it.

1.1 The problem generics solve

Before generics, a container that could hold anything held `Object`:

```
List shelf = new ArrayList(); // holds Object
shelf.add(new Book("Himu", 320));
Book b = (Book) shelf.get(0); // you promise it is a Book
Snack s = (Snack) shelf.get(0); // you also promise it is a Snack. javac allows both.
```

Both casts compile. One of them throws `ClassCastException` when the program runs — at the till, in front of a customer. The compiler could not help, because nobody ever told it what was on the shelf.

A **type parameter** tells it:

```
Shelf<Book> shelf = new Shelf<>(4);
shelf.add(new Book("Himu", 320));
Book b = shelf.get(0); // no cast: the shelf remembers what it holds
shelf.add(new Snack("Chips", 30)); // error: incompatible types. Caught now, not later.
```

That is the whole idea: **move the failure from run time to compile time.**

1.2 Bounds — promising the compiler something about T

A plain `<T>` could be anything, so inside the class you can only call `Object` methods on it. A **bound** narrows what `T` may be, and in exchange lets you call more:

```
class Shelf<T> { // T could be anything at all
    int total(T item) { return item.priceTaka(); } // error: cannot find symbol
}

class Shelf<T extends Item> { // T is at least an Item
    int total(T item) { return item.priceTaka(); } // fine, and still no cast
}
```

A bound also *keeps things out*: with the bound above, `Shelf<String>` does not compile, because a `String` is not an `Item`.

1.3 Wildcards — PECS

The two hardest characters in Java, and one sentence fixes them:

Producer Extends, Consumer Super.

If a parameter only *hands you* values, write `? extends T`. If it only *takes* values from you, write `? super T`. Both of these appear exactly once in this lab:

```
addAll(Shelf<T> shelf, List<? extends T> items) // the list only GIVES: a List<Book>
// can be poured onto a Shelf<Item>
keep(Shelf<T> shelf, Check<? super T> check) // the check only LOOKS: a Check<Item>
// works fine on a Shelf<Book>
```

1.4 Erasure — what is not there at run time

Java implements generics by *erasing* them: at run time a `Shelf<Book>` is just a `Shelf`. The type parameter is a compile-time promise, not a run-time object. So these are simply not writable, and knowing why is part of the mark:

```
new T[n] • T.class • x instanceof T
```

You will meet the first one in `Shelf`, where you want an array of `T` and cannot have it.

2. The Shop

Rahim runs a corner shop in Mohammadpur. He sells three unrelated kinds of thing:

Good	Extra field	On the shelf today
Book	author	Nondito Noroke 250, Feluda Samagra 700, Bangla Grammar 180
Snack	grams	Chanachur 40, Biscuit 25, Chips 30
SimCard	operator	GP SIM 200, Robi SIM 150

All prices are whole Taka. All three implement one interface:

```
public interface Item {
    String name();
    int priceTaka();
}
```

He needs a shelf, a way to say "there isn't one", a way to count what sold, and a couple of helpers. He needs all of that **once**, not once per kind of good. That library is **Dokan**, and you are going to finish it.

3. First, Run the Shop That Failed

An intern already tried. His code is in your starter project, in the package `intern`. Run it before you write anything:

```
./gradlew intern
```

The three pains, and where each one is

- The shelf holds Object, so it holds anything.** A snack goes onto the book shelf and `javac` says nothing; the crash comes later, as `ClassCastException: Snack cannot be cast to Book`, in a method that has nothing to do with the mistake. (ObjectShelf.java)
- One bug, fixed once, still live in the copy.** He wrote `BookShelf`, then copy-pasted it into `SnackShelf`. The removal bug was found and fixed in one of them. Run the demo: the same operation leaves `BookShelf` correct and makes `SnackShelf` throw a `NullPointerException`. (`BookShelf.java` vs `SnackShelf.java`)
- null is not an answer.** `cheapest()` returns `null` when the shelf is empty, so the next line dereferences it. "The shelf is empty" is an ordinary, expected answer and deserves a value that can say so.

Pains 1 and 2 disappear the moment you write one `Shelf<T>` instead of three shelves. Pain 3 disappears the moment you have a `Box<T>`.

4. What You Are Given, and What You Write

Open the starter project in IntelliJ. Everything already compiles and runs; the work is marked TODO.

Type	State	What it is
Item (interface)	given	The bound: a name and a price.
Check<T> (interface)	given	A yes/no question about one T.
Box<T>	TODO	A value, or nothing. Replaces null.
Pair<A,B>	1 TODO	Two type parameters. Given as a worked example except <code>swapped()</code> .
Shelf<T extends Item>	TODO	Fixed-capacity shelf of one kind of good.
Counter<T>	TODO	Counts how often each value is seen.
Shelves	TODO	Four static generic helpers.
Book, Snack, SimCard	given	The goods. Do not change them.
Sample, Main	given	The data above, and a demo of the finished shop.
intern.*	given	The failed version. Read it, then delete it.
DokanCases	given	The 27 worked cases.
DokanTest	given	41 unit tests. Do not modify.

4.1 Four commands

```
./gradlew intern # the three pains, live. Start here.
./gradlew cases # 27 worked cases: Input / Expected / Actual / Explanation
./gradlew test # the graded JUnit suite (41 tests)
./gradlew run # Main: a day at the shop
```

On Windows use `gradlew.bat`. Right now `./gradlew cases` prints **1 of 27 cases passed**. That number is your progress bar.

4.2 Build it in this order

Step	File	Turns green	The idea
1	Box.java	cases 1–4	one type parameter
	Pair.java (only swapped)	cases 5–7	two type parameters, and swapping them
2	Shelf.java	cases 8–13	a <i>bounded</i> type parameter
3	Counter.java	cases 14–19	an <i>unbounded</i> one, returning a Box
4	Shelves.java	cases 20–27	static generic methods, and PECS

5. The Five Classes, With Worked Examples

Every example below is one of the 27 cases in `./gradlew cases`, with the same number. Run the command to see them beside what your code actually produces.

5.1 Box<T> — a value, or nothing

Holds either one value of type T or nothing at all. Built with `Box.of(x)` or `Box.empty()`; ask it `isEmpty()`, `get()`, or `orElse(fallback)`. Opening an empty box is a programming mistake, so it throws; returning null would put you back where the intern was.

Cases 1–4 — Box

Input `Box.of("Chanachur").get()` **Output** Chanachur

Explanation `Box.of("...")` makes a `Box<String>`, so `get()` gives a String back. No cast: the box remembers what it holds.

Input `Box.empty().get()` **Output** `IllegalStateException`

Explanation There is no value to give. Returning null here is the bug this class exists to remove.

Input `Box.<Book>empty().orElse(GRAMMAR).name()` **Output** `Bangla Grammar`

Explanation The fallback must be a Book, because the box is a `Box<Book>`. Try passing a `Snack` and read what javac says.

5.2 `Pair<A,B>` — two type parameters at once

Two values of two possibly different types, kept together: `first()`, `second()`, and `swapped()`. Everything but `swapped()` is written for you, as a worked example of the style the rest of your code should follow.

Cases 5–7 — Pair

Input `new Pair<>("Chanachur", 40)` **Output** `(Chanachur, 40)`

Explanation A is String, B is Integer. They are independent — nothing forces them to be the same type.

Input `new Pair<>("Chanachur", 40).swapped()` **Output** `(40, Chanachur)`

Explanation A `Pair<String,Integer>` became a `Pair<Integer,String>`. Look at the return type of `swapped()`: the type parameters swap places along with the values, and the compiler holds you to it.

Input `pair.swapped().first() + 1` **Output** `41`

Explanation After the swap, `first()` is an Integer, so arithmetic on it compiles. Before the swap it was a String and this line would not.

5.3 `Shelf<T extends Item>` — the bounded one

A shelf of fixed capacity holding items of one kind: `add` (false when full), `get`, `size`, `capacity`, `isFull`, `isEmpty`, `contains(name)`, `remove(name)`, `totalPriceTaka()` and `items()` (a list the caller cannot use to change the shelf).

The erasure moment

You will want `new T[capacity]`. You cannot have it — at run time there is no T to make an array of. Use an `ArrayList<T>` and keep the capacity in an `int`. Say so in your README; that is one of the marks.

Cases 8–13 — Shelf

Input shelf of capacity 4; add the three books, then `totalPriceTaka()`

Output `1130`

Explanation `250 + 700 + 180`. This method can only call `priceTaka()` because of the bound `<T extends Item>`. Delete the bound and it stops compiling — it is worth trying once.

Input add a 4th book, then a 5th

Output `4th=true full=true 5th=false size=4`

Explanation Being full is ordinary, so it is a false, not an exception.

Input `remove("Feluda Samagra")` from the three books

Output `removed=true size=2 total=430`

Explanation `250 + 180`. Close the gap you leave behind — the intern's `SnackShelf` blanks the slot, keeps the count, and then throws.

Input `Shelf<Snack>` with all three snacks

Output `size=3 total=95`

Explanation `40 + 25 + 30`. Not one line of `Shelf` was copied or changed to make this work. The intern needed a whole second file — and a second bug.

5.4 Counter<T> — the unbounded one

Counts how many times each value has been seen: `add(value)`, `count(value)`, `distinct()`, and `mostCommon()` which returns a `Box<T>`. Note that `T` has *no bound*: this class does not need to know what an item is, so it does not ask. Adding a bound you do not need is a real design mistake, not a harmless extra.

Cases 14–19 — Counter

Input `add Chanachur, Biscuit, Chanachur, Chips, Chanachur;` then `count("Chanachur")`

Output 3 (and `count("Chocolate")` is 0, and `distinct()` is 3)

Explanation Something never seen counts zero. Not null, not an exception.

Input `mostCommon()` **Output** `Box(Chanachur)`

Explanation A `Counter<String>` gives back a `Box<String>`. The type parameter travels from one generic class into another.

Input `new Counter<String>().mostCommon()` **Output** `Box(empty)`

Explanation Nothing sold yet. No special case, no null, no crash — the empty box already means exactly this.

Input `Counter<Book>`: `add NONDITO` twice and `FELUDA` once

Output `count(NONDITO)=2 best=Nondito Noroke (250 Tk)`

Explanation Strings on one line, Books on the next, one class for both.

5.5 Shelves — static generic methods

Four helpers that work on any shelf of any good. Each has its `<T>` on the *method*, not the class, which is why you write `Shelves.cheapest(books)` and never `Shelves.<Book>cheapest(books)` — the compiler works the type out from the argument. That is called **inference**.

```
static <T extends Item> Box<T> cheapest(Shelf<T> shelf)
static <T extends Item> List<T> keep(Shelf<T> shelf, Check<? super T> check)
static <T extends Comparable<T>> T max(List<T> values)
static <T extends Item> int addAll(Shelf<T> shelf, List<? extends T> items)
```

Cases 20–27 — Shelves

Input `Shelves.cheapest(bookShelf)` **Output** `Bangla Grammar`

Explanation And on an empty shelf it is `Box(empty)` — the intern returned null here and the next line threw.

Input `Shelves.keep(bookShelf, item -> item.priceTaka() < 300)`

Output `[Nondito Noroke, Bangla Grammar]`

Explanation The check is a `Check<Item>` but the shelf is a `Shelf<Book>`. That only compiles because `keep` takes `Check<? super T>` — *consumer super*. A test that works for any item obviously works for a book.

Input `Shelves.max(List.of(40, 25, 30))` and `Shelves.max(List.of("Chanachur", "Biscuit", "Chips"))`

Output 40 and Chips

Explanation One method, two completely different types, no overloading. The bound `<T extends Comparable<T>>` is what allows `compareTo` at all.

Input `Shelves.addAll(new Shelf<Item>(4), BOOKS)`

Output `added=3 total=1130`

Explanation The list only hands books over, so `addAll` takes `List<? extends T>` — *producer extends*. With a plain `List<T>` this line would not compile, because a `List<Book>` is not a `List<Item>`.

Input `Shelf<Book>`, `Shelf<Snack>`, `Shelf<SimCard>`

Output `books=3 snacks=3 sims=2`

Explanation One class, written once. The intern wrote two and would have needed a third — and he fixed his bug in only one of them.

6. The Marks You Cannot Earn by Running Anything

checks/CompileFailWitnesses.java holds four commented-out lines. Each is a mistake the intern's shop compiled happily and then failed on at run time. With correct type parameters, **javac must refuse them**. Uncomment them *one at a time*, run `./gradlew compileJava`, and confirm the build fails with the stated error. Comment them back out before you submit.

```
bookShelf.add(Sample.CHANACHUR);
    // error: incompatible types: Snack cannot be converted to Book
Shelf<String> nonsense = new Shelf<>(4);
    // error: type argument String is not within bounds of type-variable T
Book wrong = Box.of(Sample.CHANACHUR).get();
    // error: incompatible types: Snack cannot be converted to Book
Pair<String, Integer> same = new Pair<>("Chanachur", 40).swapped();
    // error: incompatible types: Pair<Integer,String> cannot be converted to Pair<String,Integer>

Book book = bookShelf.get(0); // this one MUST still compile
```

If any of the four compiles, a type parameter is missing — and you lose those marks even if all 27 cases and all 41 tests are green.

7. Hard Constraints

Read these before you write a line — they are most of the mark

1. **No raw types, anywhere.** Writing `Shelf` where you mean `Shelf<Book>` is a raw type. After deleting the `intern` package, `./gradlew strict` (which compiles with `-Xlint:unchecked,rawtypes -Werror`) must pass.
2. **No Object-typed fields or parameters** in `dokan`, and **no cast** to `Book`, `Snack` or `SimCard` anywhere at all. If you are casting, a type parameter is missing.
3. **Do not change the given signatures.** The wildcards in `Shelves` are the marked part, and unit tests pin them.
4. **Do not add bounds you do not need.** `Counter<T>` must stay unbounded; `Counter<T extends Item>` would be wrong even though every test still passed.
5. **Erasure.** No `new T[n]`, `T.class`, or `x instanceof T`. Your `README.txt` must say where you wanted one and what you did instead.
6. **Nothing in dokan may import anything from shop.** The toolkit must not know that books exist. This is checkable by eye, and it is the premise of the whole lab.
7. **Prices are whole Taka in an int.** No doubles, no rounding.

8. Submission

- Submit your `.java` sources with the `intern` package deleted and the four witness lines commented out, so the project builds.
- Include a `README.txt` with: your team members and roll numbers; one sentence per type saying whether it is a class or an interface and why; every type parameter you wrote with one line on why it is bounded the way it is or *why it needs no bound*; and your answer to the erasure question.
- Partial credit applies — submit whatever compiles. A project where `./gradlew cases` reports 20 of 27 is worth far more than one that does not build.